Evaluating Cross-Architecture Robustness in Adversarial Neural Cryptography

CS 180H Research Proposal | Department of Computer Science | San José State University

Kunj Shah | Student ID: 016970226 | Faculty advisor: Professor Mark Stamp
kunj.shah@sjsu.edu | kunjcr2.github.io

1. Project summary

Adversarial neural cryptography asks whether neural networks can learn keyed communication through a contest among Alice (the sender), Bob (the receiver), and Eve (an eavesdropper) [1]. Alice maps a binary message M and shared key K to ciphertext C; Bob reconstructs M from (C, K); Eve tries to recover M from C alone. Alice and Bob may appear successful when the Eve used during training guesses near chance, yet a newly trained attacker may extract information from the same ciphertext. This project will measure that gap across attacker architectures, model sizes, and training budgets. Its goal is a reproducible evaluation of learned communication, *not* a replacement for established cryptography or a claim of formal secrecy.

2. Research question and motivation

The central question is: *How well does an Alice–Bob pair trained against one Eve architecture resist fresh attackers with different architectures?* Two related questions are whether attacker capacity changes that result, and whether a longer optimization budget is necessary to reveal a capacity effect. These distinctions matter because a failed in-training Eve is only one attempted attack. Abadi and Andersen retrained a fresh Eve after freezing Alice and Bob [1]; subsequent work has examined stronger security objectives, leakage, and multiple adversaries [2, 3, 4]. Architecture changes have also been studied in complete neural cryptography systems [8]. This proposal focuses on a controlled attack-transfer comparison against the *same frozen* Alice–Bob pair, so the effect of a new attacker can be distinguished from a change to the communication system.

3. Preliminary work

I have implemented 32-bit Alice–Bob–Eve experiments in PyTorch and recorded their protocols and outcomes in the project repository. A pilot capacity-matched 4×4 matrix crossed CNN, multilayer perceptron (MLP), bidirectional LSTM, and Transformer training Eves with fresh attackers from those four families. That pilot used one pair seed, so it motivates replication rather than a general architecture ranking.

Later experiments found that attack architecture and optimization time interact. On one reliably communicating pair trained against an MLP Eve, fresh MLP bit accuracy rose from 55.73% at roughly 5,000 parameters to 63.83% at roughly 80,000 parameters after 20,000 attacker updates; CNN attack accuracy was comparatively flat. A three-pair-seed comparison found that the 80,000-parameter fresh MLP outperformed a 5,000-parameter MLP on every successful MLP-Eve and CNN+MLP-Eve pair. Across the three MLP-Eve pairs, the mean held-out bit accuracies were 58.03% and 68.85%, respectively. These are empirical attack results, not security guarantees; they also show why attacker size cannot be assessed without enough training. A CNN-only training condition failed Bob communication on two of three seeds, illustrating that communication reliability must be checked before privacy results are interpreted. The full run settings, notebooks, caveats, and figures are documented in the existing project materials.

4. Proposed method

1. **Establish comparable systems.** Use independently sampled 32-bit messages and keys, the existing CNN Alice/Bob implementation, and a fixed training protocol. Train separate Alice–Bob pairs against CNN, MLP, bidirectional LSTM, or Transformer Eves. Record pair seeds, architecture parameters, compute budgets, and checkpoints. A small one-time-pad-style control and a deliberately leaky transform will check that the attack pipeline behaves as expected.
2. **Gate on communication.** Evaluate Bob on fresh, held-out message–key pairs. Report both bit accuracy and exact 32-bit message accuracy. Apply a prespecified reliability threshold (at least 96% bit accuracy on validation and test) before comparing attacker results, and report how often each training condition fails this gate. Exact-message accuracy will remain visible because high bit accuracy can conceal poor complete-message

recovery.

3. **Freeze and attack.** Freeze each accepted Alice–Bob pair. Train fresh ciphertext-only CNN, MLP, bidirectional LSTM, and Transformer attackers from independent initializations. Use comparable data splits and update budgets, select checkpoints on validation data, and evaluate only once on a separate test split. Compare approximately matched parameter counts; the primary capacity contrast is about 5,000 versus 80,000 parameters, with intermediate sizes where compute permits. Measure attackers after 5,000 and 20,000 updates to expose training-time effects.
4. **Repeat and analyze.** Target at least five independently trained pair seeds for the main conditions and three attacker initializations per architecture/size condition. Prioritize a complete, replicated CNN/MLP/Transformer comparison; include the bidirectional LSTM in the full matrix as compute allows. Report mean and variation *across pairs* separately from variation *across attacker starts*. Do not count an unstable or failed Alice–Bob pair as evidence of privacy. Investigate the anomalous CNN-only pilot by saving ciphertext samples and checkpoints, testing key dependence, and retraining a fresh same-architecture Eve.

The primary outcome is held-out plaintext-bit accuracy for a fresh Eve; 50% is the random-guessing baseline for balanced independent bits. Secondary outcomes are exact-message recovery, Hamming error, Bob's two accuracy measures, and training success rate. The analysis will identify attacker advantages over chance and compare cross-architecture cells within each frozen pair. Parameter count is only an approximate capacity match: architectures also differ in optimization behavior and computation. Therefore, the study will report actual parameter counts and measured training budgets rather than treating equal counts as equal capability.

5. Scope, resources, and deliverables

The core study uses the existing PyTorch notebooks and result log, with NumPy and plotting tools for analysis. GPU access through available university resources or a cloud notebook (the pilots used an NVIDIA A100) will support repeated runs; the experiment matrix will be reduced in a documented order if that compute is unavailable. No private dataset is needed: plaintexts and keys are freshly generated random bits. Versioned configurations, seeds, held-out split definitions, checkpoints, and raw per-run metrics will make the results reproducible.

The CS 180H deliverables are: (1) a documented benchmark and attack harness; (2) a replicated attack-transfer table with communication and attack metrics; (3) capacity-versus-training-time plots with uncertainty across seeds; (4) an analysis of failed communication runs and the CNN-only anomaly; and (5) a written research report and presentation describing the findings and limitations. The initial milestone is to lock the evaluation protocol and reproduce the existing pilots. The next is to complete the replicated core matrix. The final phase is analysis, targeted follow-up attacks, and the report. An optional extension is ensemble Eve training or a chosen-plaintext distinguishing attack, motivated by prior stronger-threat-model work [2]; neither is required for the core deliverables.

6. Expected contribution and limits

The expected contribution is a controlled, reusable way to test whether a learned communication scheme has merely adapted to its training attacker. The project may find cross-architecture leakage, architecture-specific capacity effects, or more robust behavior under some conditions. Any of these outcomes would clarify how neural cryptography experiments should be evaluated. Failure of the tested Eves to recover plaintext would establish only empirical resistance to those Eves and budgets. It would not prove semantic security or resistance to other learned or analytical attacks. The broader literature includes topology changes, longer messages, asymmetric settings, multi-attacker systems, and a recent survey of open evaluation challenges [5, 6, 7, 9, 10]; those topics inform the discussion but are outside the controlled core study.

References

[1] M. Abadi and D. G. Andersen, "Learning to Protect Communications with Adversarial Neural Cryptography," 2016. https://arxiv.org/abs/1610.06918.

[2] M. Coutinho et al., "Learning Perfectly Secure Cryptography to Protect Communications with Adversarial Neural Cryptography," *Sensors*, 2018.

[3] L. Zhou et al., "Security Analysis and New Models on the Intelligent Symmetric Key Encryption," *Computers & Security*, 2019.

[4] Z. Li et al., "Information Encryption Communication System Based on the Adversarial Networks Foundation," *Neurocomputing*, 2020.

[5] Y. Zhu, D. V. Vargas, and K. Sakurai, "Neural Cryptography Based on the Topology Evolving Neural Networks," *CANDAR Workshops*, 2018.

[6] X. Duan et al., "Optimization of Encrypted Communication Length Based on Generative Adversarial Network," *IEEE BDAI*, 2021.

[7] I. Meraouche et al., "Learning Asymmetric Encryption Using Adversarial Neural Networks," *Engineering Applications of Artificial Intelligence*, 2023.

[8] J. H. Tyler, M. K. M. Fadul, and D. R. Reising, "Transformers—Messages in Disguise," *Machine Learning with Applications*, 2025. https://doi.org/10.1016/j.mlwa.2025.100669.

[9] Y. Wang et al., "AdvOT: Oblivious Transfer Based on Generative Adversarial Networks Against Multiple Attackers," *Neurocomputing*, 2025.

[10] Y. Zhong, J. Gu, and X. Yu, "Neural Cryptography: Synergy and Conflict Between Neural Networks and Cryptographic Systems," *Neurocomputing*, 2026. https://doi.org/10.1016/j.neucom.2026.133084.